



Преподаватель:

Коляда

Никита Владимирович

Обратная связь:

- сообщения на inStudy

ПОДКЛЮЧЕНИЕ NODE.JS К MYSQL

—

ПЕРЕМЕННЫЕ ОКРУЖЕНИЯ И БИБЛИОТЕКА MYSQL2

2026 Г.

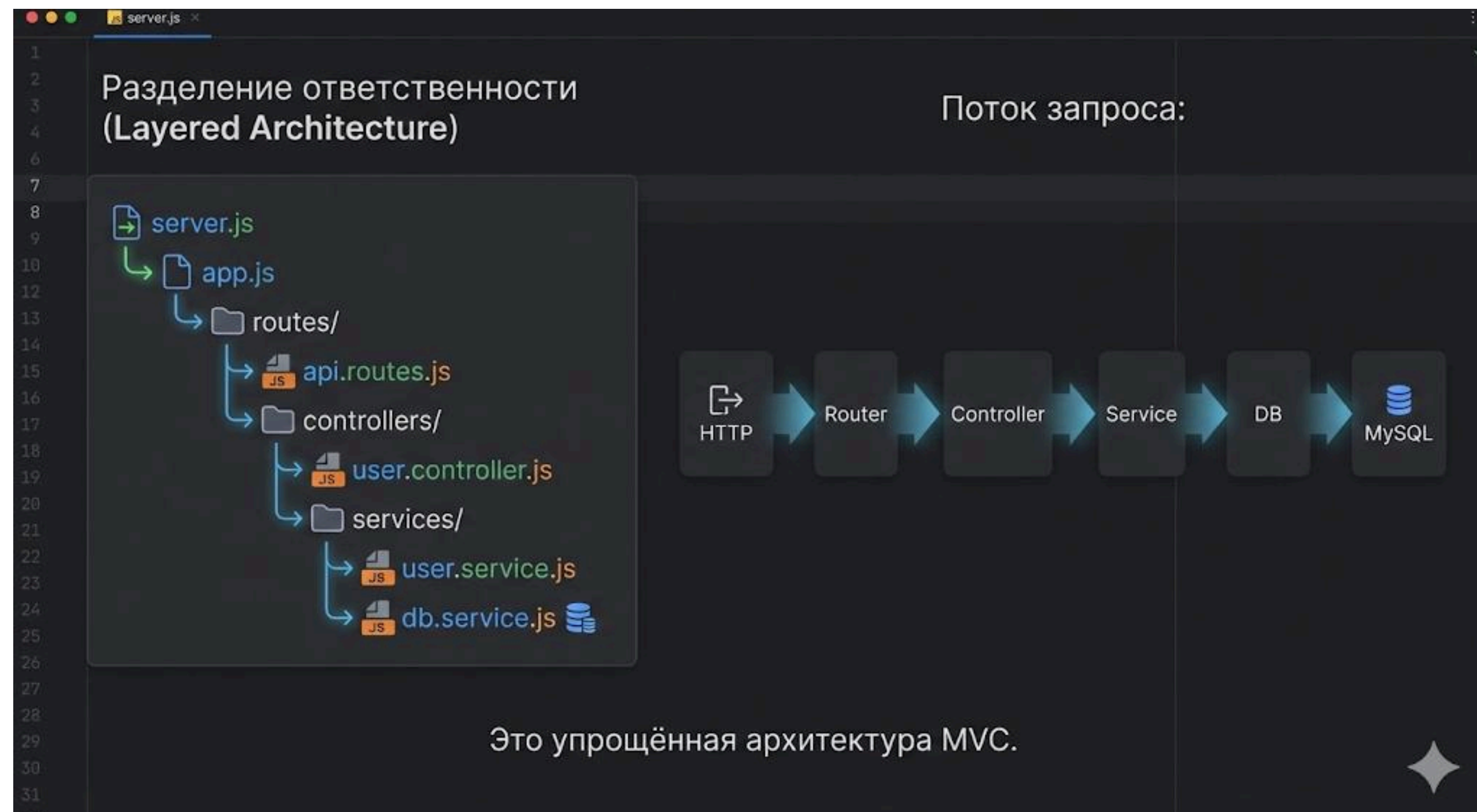


ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

ОБЩАЯ АРХИТЕКТУРА ПРОЕКТА

РАЗДЕЛЕНИЕ ОТВЕТСТВЕННОСТИ



1. Структура:

- a. `server.js` → Точка входа
- b. `app.js` → Настройка Express
- c. `routes/` → Маршруты
- d. `controllers/` → HTTP-логика
- e. `services/` → Бизнес-логика
- f. `db.service.js` → Подключение к БД

2. Поток запроса:

- a. `HTTP` → `Router` → `Controller` → `Service` → `DB` → `MySQL`

ЧТО ТАКОЕ ПЕРЕМЕННЫЕ ОКРУЖЕНИЯ?

ПОЧЕМУ DOTENV ЗАГРУЖАЕТСЯ ПЕРВЫМ?

1. Конфигурация вне кода

2. Переменные окружения — это способ хранить:

- логины
- пароли
- порты
- режим работы
- конфигурацию БД

Конфигурация вне кода

Переменные окружения — это способ хранить:

-  логины
-  пароли
-  порты
-  режим работы
-  конфигурацию БД

```
.env
1
2 PORT=3000
3 NODE_ENV=development
4 DB_HOST=localhost
5 DB_USER=root
6 DB_PASSWORD=1234
7 DB_NAME=storedb
8 DB_CONNECTION_LIMIT=5
9
10
11
12
13
14
15
```

```
assets
├── example
│   ├── node_modules
│   │   └── library root
│   └── src
│       ├── controllers
│       ├── middlewares
│       ├── routes
│       ├── services
│       ├── app.js
│       └── server.js
├── .env
├── .env.example
├── package.json
├── package-lock.json
└── README.md
10
11 # --- Настройки сервера ---
12 PORT=3000
13 NODE_ENV=development
14
15 # --- Настройки базы данных MySQL ---
16 DB_HOST=localhost
17 DB_USER=root
18 DB_PASSWORD=
19 DB_NAME=storedb
20 DB_CONNECTION_LIMIT=5
21
```


ПОДКЛЮЧЕНИЕ БИБЛИОТЕКИ DOTENV

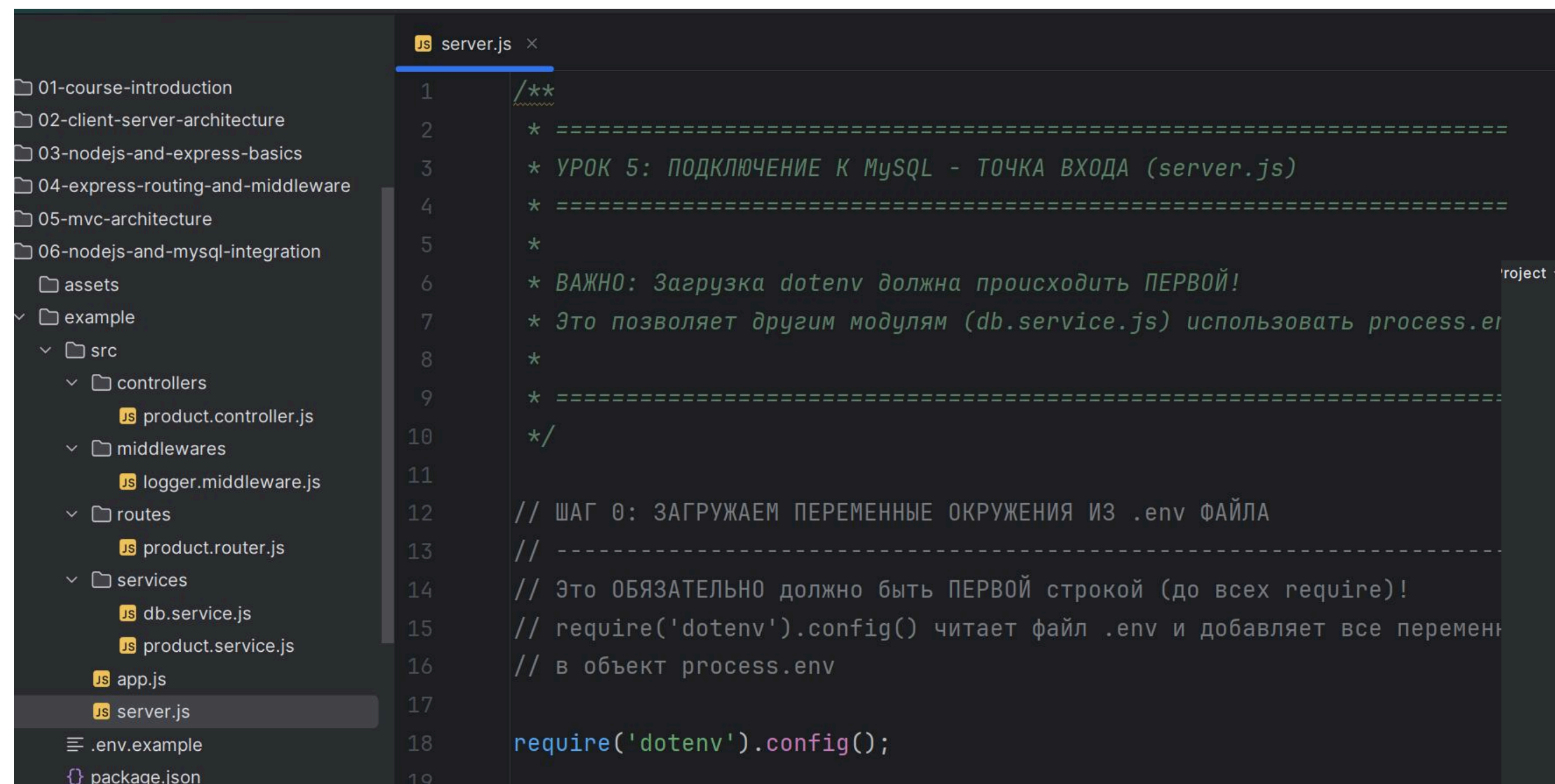
ПОЧЕМУ DOTENV ЗАГРУЖАЕТСЯ ПЕРВЫМ?

1. Устанавливаем библиотеку **dotenv**

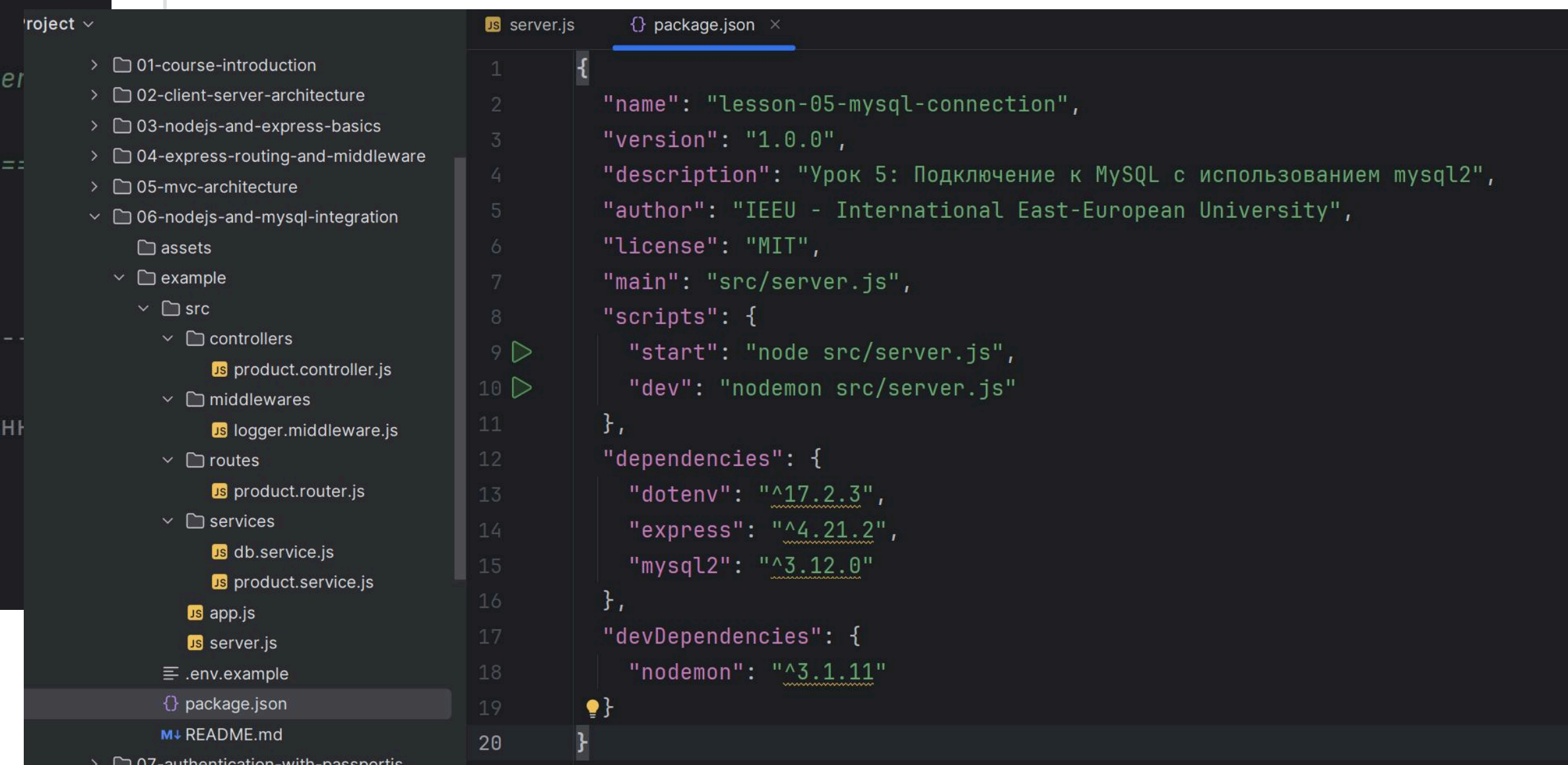
a. **npm install dotenv**

2. Как подключить в проекте:

- **.env** загружается в **process.env**
- Это должно быть ДО всех require
- Иначе другие модули не увидят переменны



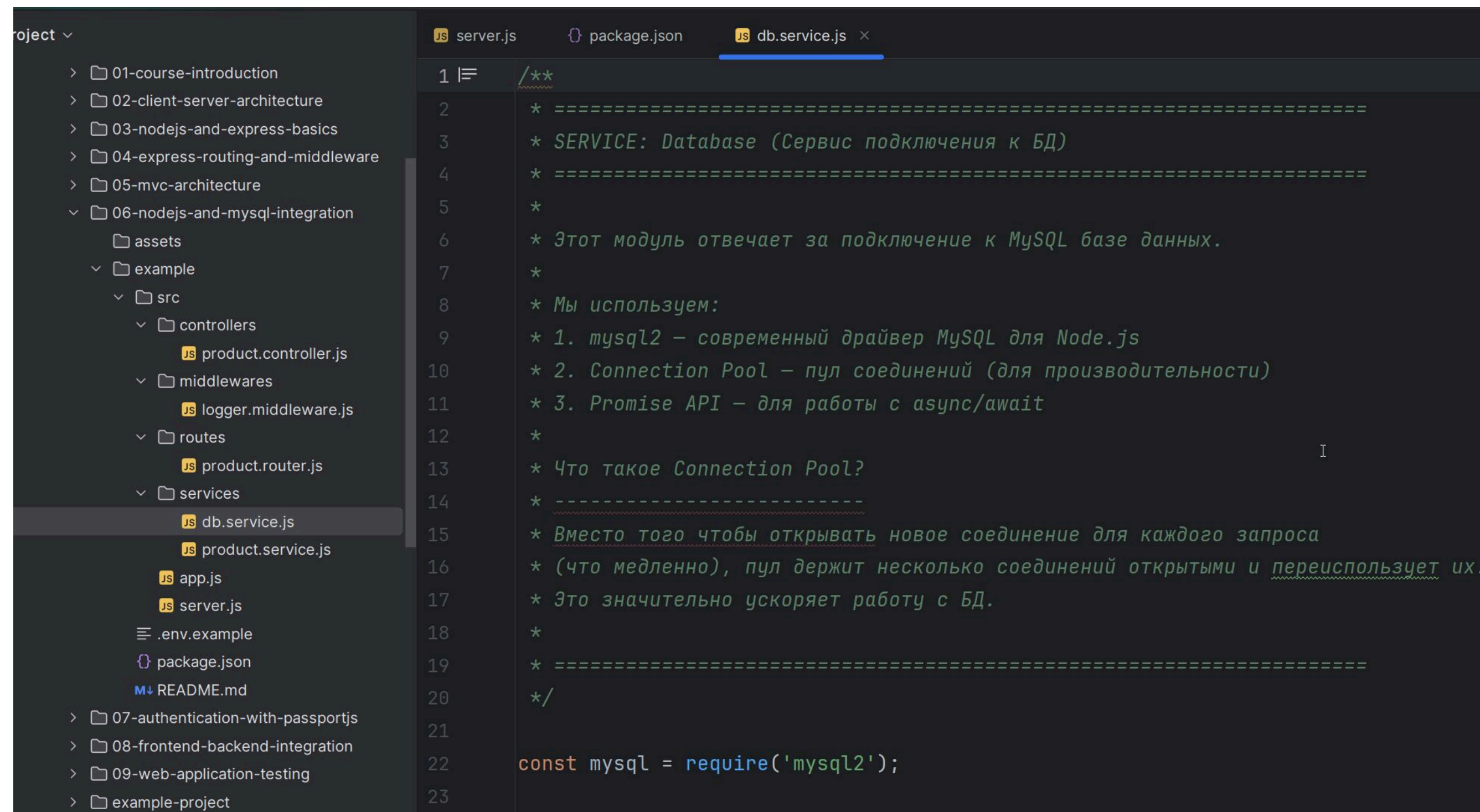
```
1  /**
2  * =====
3  * УРОК 5: ПОДКЛЮЧЕНИЕ К MySQL - ТОЧКА ВХОДА (server.js)
4  * =====
5  *
6  * ВАЖНО: Загрузка dotenv должна происходить ПЕРВОЙ!
7  * Это позволяет другим модулям (db.service.js) использовать process.env
8  *
9  * =====
10 * /
11
12 // ШАГ 0: ЗАГРУЖАЕМ ПЕРЕМЕННЫЕ ОКРУЖЕНИЯ ИЗ .env ФАЙЛА
13 // -----
14 // Это ОБЯЗАТЕЛЬНО должно быть ПЕРВОЙ строкой (до всех require)!
15 // require('dotenv').config() читает файл .env и добавляет все переменны
16 // в объект process.env
17
18 require('dotenv').config();
19
```



```
1  {
2    "name": "lesson-05-mysql-connection",
3    "version": "1.0.0",
4    "description": "Урок 5: Подключение к MySQL с использованием mysql2",
5    "author": "IEEU - International East-European University",
6    "license": "MIT",
7    "main": "src/server.js",
8    "scripts": {
9      "start": "node src/server.js",
10     "dev": "nodemon src/server.js"
11   },
12   "dependencies": {
13     "dotenv": "^17.2.3",
14     "express": "^4.21.2",
15     "mysql2": "^3.12.0"
16   },
17   "devDependencies": {
18     "nodemon": "^3.1.11"
19   }
20 }
```


MYSQL2 — ДРАЙВЕР MYSQL ДЛЯ NODE.JS

НИЗКОУРОВНЕВЫЙ ДОСТУП К БАЗЕ ДАННЫХ



1. mysql2 — это официальный Node.js-драйвер для работы с MySQL через TCP-протокол.
2. Он реализует клиентскую часть MySQL-протокола и позволяет выполнять SQL-запросы из JavaScript.

3. Установка:

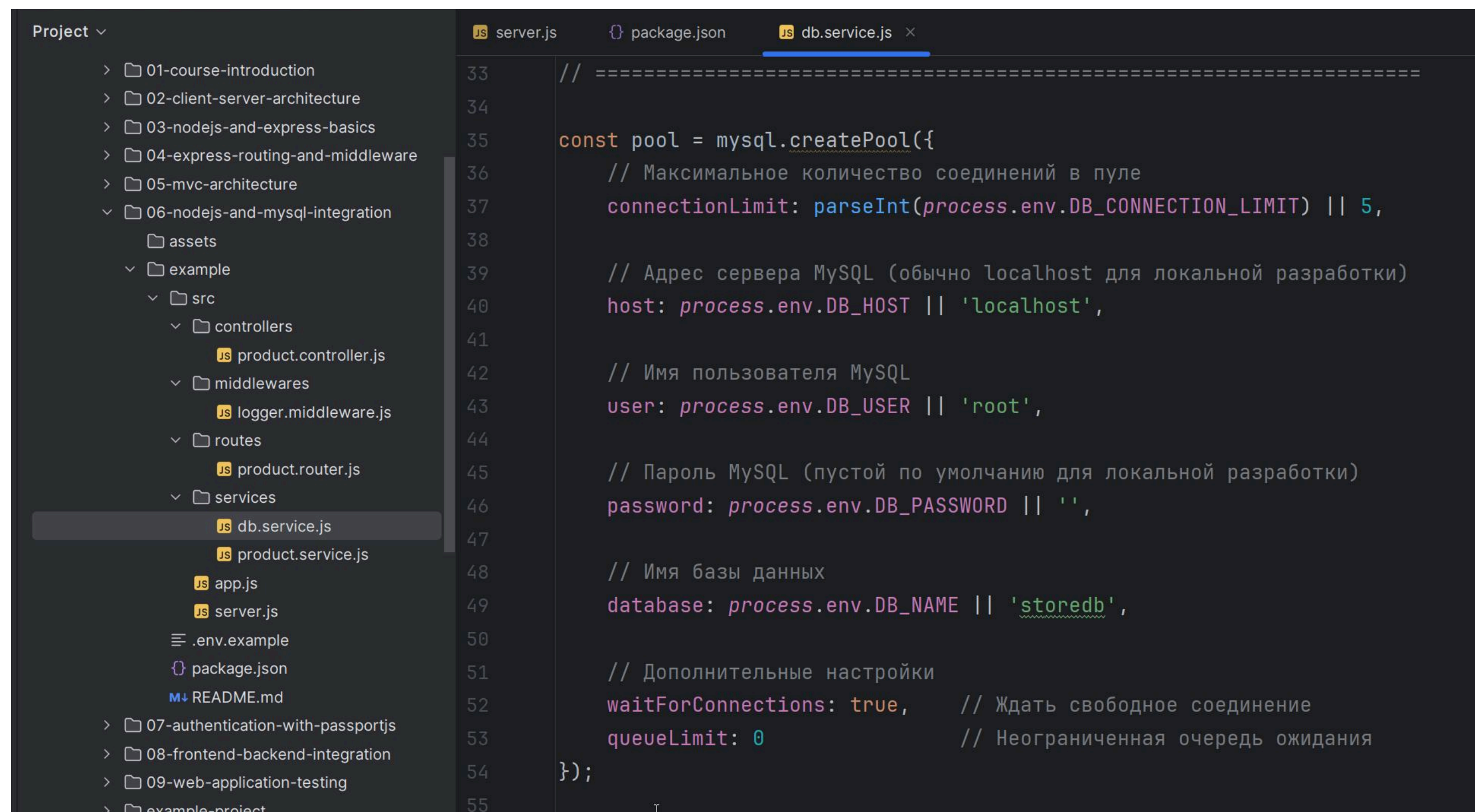
a. npm install mysql2

4. Подключаем библиотеку в файле

db.service.js

CONNECTION POOL

НАСТРОЙКА СОЕДИНЕНИЯ



```
33 // =====
34
35 const pool = mysql.createPool({
36   // Максимальное количество соединений в пуле
37   connectionLimit: parseInt(process.env.DB_CONNECTION_LIMIT) || 5,
38
39   // Адрес сервера MySQL (обычно localhost для локальной разработки)
40   host: process.env.DB_HOST || 'localhost',
41
42   // Имя пользователя MySQL
43   user: process.env.DB_USER || 'root',
44
45   // Пароль MySQL (пустой по умолчанию для локальной разработки)
46   password: process.env.DB_PASSWORD || '',
47
48   // Имя базы данных
49   database: process.env.DB_NAME || 'storedb',
50
51   // Дополнительные настройки
52   waitForConnections: true, // Ждать свободное соединение
53   queueLimit: 0             // Неограниченная очередь ожидания
54 });
55
```

1. Создание пула соединений (поддержка нескольких соединений - удобно)

происходит с помощью метода

a. `mysql.createPool(...)`

2. Пул:

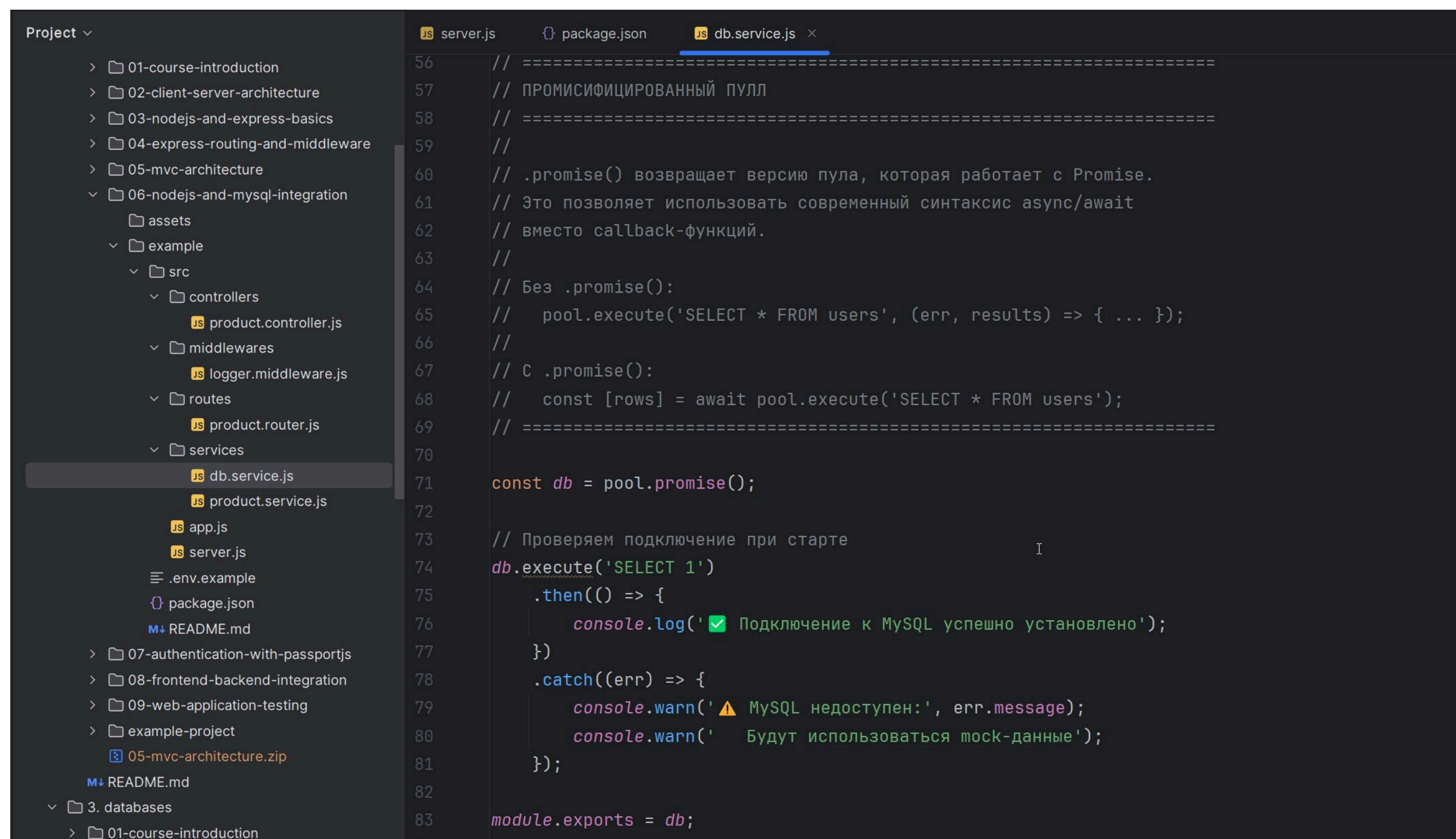
- держит несколько соединений открытыми
- переиспользует их
- управляет очередью запросов

3. Разбор ключевых параметров:

- **`connectionLimit`** — максимум соединений
- **`waitForConnections`** — ждать освобождения
- **`queueLimit`** — лимит очереди

ПРОМИСИФИЦИРОВАННЫЙ ПУЛ (PROMISE POOL)

РАБОТА С MYSQL ЧЕРЕЗ ASYNC/AWAIT



```
56 // =====
57 // ПРОМИСИФИЦИРОВАННЫЙ ПУЛ
58 // =====
59 //
60 // .promise() возвращает версию пула, которая работает с Promise.
61 // Это позволяет использовать современный синтаксис async/await
62 // вместо callback-функций.
63 //
64 // Без .promise():
65 // pool.execute('SELECT * FROM users', (err, results) => { ... });
66 //
67 // С .promise():
68 // const [rows] = await pool.execute('SELECT * FROM users');
69 // =====
70
71 const db = pool.promise();
72
73 // Проверяем подключение при старте
74 db.execute('SELECT 1')
75   .then(() => {
76     console.log('✅ Подключение к MySQL успешно установлено');
77   })
78   .catch((err) => {
79     console.warn('⚠️ MySQL недоступен:', err.message);
80     console.warn('Будут использоваться мошк-данные');
81   });
82
83 module.exports = db;
```

1. В mysql2 пул соединений по умолчанию работает через **callback-API**.

2. Чтобы использовать современный синтаксис **async/await**, необходимо получить **Promise-версию пула**.

3. Зачем это нужно?

а. Избавление от **callback-hell**. Вместо вложенных функций — линейный код.

б. Читаемость. Код выглядит синхронным, но выполняется асинхронно.

с. Централизованная обработка ошибок

AWAIT DB.EXECUTE()

ВЫПОЛНЕНИЕ SQL-ЗАПРОСА ЧЕРЕЗ PROMISE POOL

```
const [rows] = await db.execute(  
  'SELECT * FROM products'  
);
```

```
const [rows] = await db.execute(  
  'SELECT * FROM products WHERE product_id = ?',  
  [productId]  
);
```

1. В контексте mysql2 метод **execute()** используется для выполнения подготовленных (prepared) **SQL-запросов** через промисифицированный пул.

2. Общий синтаксис

a. `const [rows, fields] = await db.execute(sql, values);`

b. Где:

- **sql** — строка SQL-запроса
- **values** — массив параметров (если есть ?)
- **rows** — результат запроса
- **fields** — метаданные колонок

3. Пример с параметрами (Prepared Statement)

- a. **?** — placeholder
- b. Значение из массива безопасно подставляется
- c. Данные экранируются автоматически
- d. Это защита от SQL Injection.

ЧТО ВОЗВРАЩАЕТ EXECUTE()

ВЫПОЛНЕНИЕ SQL-ЗАПРОСА ЧЕРЕЗ PROMISE POOL

1. SELECT

```
const [rows, fields] = await db.execute(sql);
```

2. INSERT

```
const [result] = await db.execute(  
  'INSERT INTO products (product_name, product_price) VALUES (?, ?)',  
  [name, price]  
);
```

3. UPDATE

```
const [result] = await db.execute(  
  'UPDATE products SET product_price = ? WHERE product_id = ?',  
  [price, id]  
);
```

1. В зависимости от типа запроса **result** будет содержать:

2. SELECT запрос

a. rows — массив результатов

b. fields — метаданные колонок

3. INSERT запрос

a. insertId — ID новой записи

b. affectedRows — сколько строк изменено

c. warningStatus — код ошибки

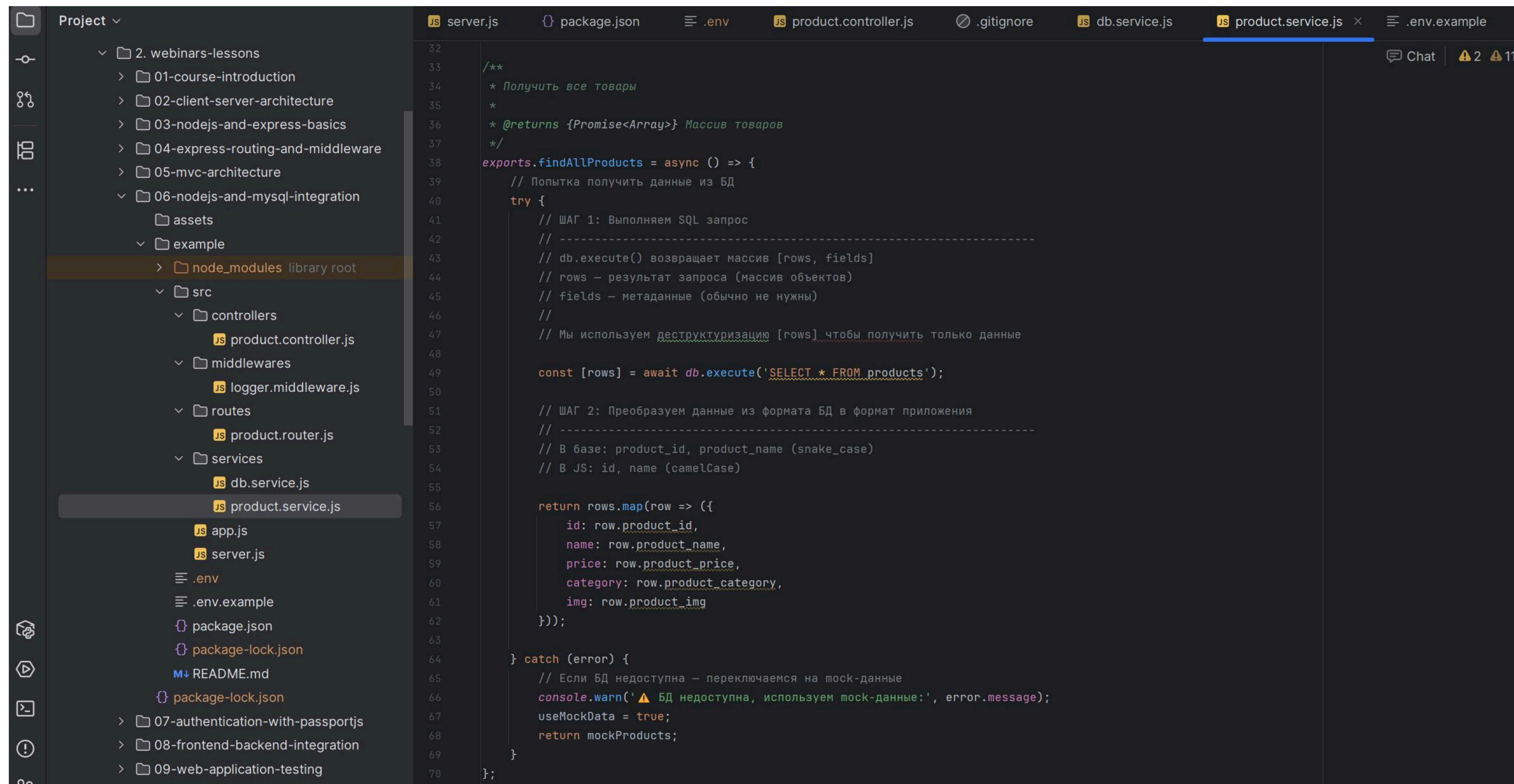
4. UPDATE and DELETE запрос

a. affectedRows — сколько строк изменено

b. warningStatus — код ошибки

ПРИМЕР ПОЛУЧЕНИЯ СПИСКА ТОВАРОВ ИЗ БД

ПРАКТИКА



ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ ПОЛЬЗОВАТЕЛЯ